

重点回答

SGLang RadixAttention / RadixCache、分页 KV 与 vLLM PagedAttention

1. RadixAttention / RadixCache 解决什么问题？

不同请求或多轮对话中，提示词的前缀可能相同。每次都从头做 Prefill，就会重复计算这部分 KV。SGLang 用基数树保存 token 前缀和对应的

KV。新请求先匹配已有的最长前缀，命中后只计算剩余内容。缓存满时可以按 LRU 淘汰旧内容，调度时也可以考虑请求命中缓存的可能性。我理解，RadixAttention 主要是为了避免相同前缀反复计算。[1]

2. page-sized KV cache 与 prefix reuse 在哪里？

分页 KV cache 负责 KV 在显存中的存放和读取。Prefill 写入输入对应的 KV，Decode 生成新 token 时继续读取并追加 KV。prefix reuse 则在请求进入 Prefill 前，先检查已有的前缀 KV，让 Prefill 只处理未命中的部分。基数树管理前缀与 KV 页的对应关系。这两种机制可以配合使用，各自负责的事情不同。[1]

3. vLLM PagedAttention：联系与差异

vLLM 的 PagedAttention 将 KV 分成固定大小的块，通过逻辑块与物理块的映射按需分配显存。这样就不必为每个请求预留一整段连续空间，既能减少碎片和浪费，也能共享块。[2] 我的理解是，SGLang 论文中的 RadixAttention 侧重于查找和复用已有的最长前缀，PagedAttention 侧重于 KV 块的显存管理和读取。vLLM 后来也加入了自动前缀缓存，通过块哈希实现复用，因此不能说它没有前缀复用功能。[3]

[1] Zheng 等，SGLang, NeurIPS 2024, §3:
<https://arxiv.org/html/2312.07104>

[2] Kwon 等，PagedAttention, SOSP 2023, §3:
<https://arxiv.org/html/2309.06180>

[3] vLLM Automatic Prefix Caching 设计文档:
https://docs.vllm.ai/en/latest/design/prefix_caching/